International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 7 – Test on OOP Concepts (Set A)

Date: 11.09.2026 **Semester:** B.Tech 3rd Semester **Group:** 1

Instructions

- Write all programs in **C++**.
- Use the OOP concepts specified in each question appropriately.
- Use **pointers** and dynamic memory allocation wherever required.
- Properly release dynamically allocated memory using `delete` or `delete[]`.
- Display the output in a clear and readable format.

1. **Smart Locker Allocation**
   A hostel wants to manage lockers whose number is decided at runtime. Create a class `Locker` containing locker number, occupied status, and a dynamically allocated access-code array.

   Create n lockers dynamically. Implement overloaded functions `setCode()` such that one version accepts a complete code and another changes the code at a specified position. Access the lockers using pointers and correctly release all dynamically allocated memory.

2. **Drone Battery Monitor**
   Create a class `Drone` containing drone ID, battery percentage, and flight hours. Implement overloaded `update()` functions: one updates only the battery percentage, while the other updates both battery percentage and flight hours.

   Write a friend function `compareBattery()` that receives two `Drone` objects and reports which drone has the higher battery level. Create the drones dynamically and access them using pointers.

3. **Movie Queue Display**
   A cinema maintains a waiting queue whose size is entered by the user. Create a class `QueueDisplay` containing the queue size and a dynamically allocated array of customer IDs.

   Provide functions to insert IDs and display them. Write a friend function that receives two `QueueDisplay` objects and exchanges their complete queue information. Create an array of objects dynamically and release the allocated memory correctly.

4. **Laboratory Instrument Access**
   Create a class `Instrument` containing instrument ID, name, and a private access level. Create another class `LabSupervisor` responsible for checking and modifying the access level.

   The supervisor must be able to access and modify the private members of `Instrument` without making them public. Demonstrate the appropriate OOP mechanism. Create the instrument dynamically and release it properly.

5. **E-Wallet Transaction Record**
   Create a class `Wallet` containing wallet ID, current balance, and a dynamically allocated array of transaction amounts.

   Implement overloaded `transaction()` functions: one accepts an amount and performs a deposit or withdrawal depending on its sign, while another accepts an amount and a transaction type character (`'D'` or `'W'`).

   Write a friend function that receives two wallets and determines which has the larger balance without using a public getter for the balance. Create the wallets dynamically and release the memory correctly.